

Deep Learning (IST, 2024-25)

Homework 1

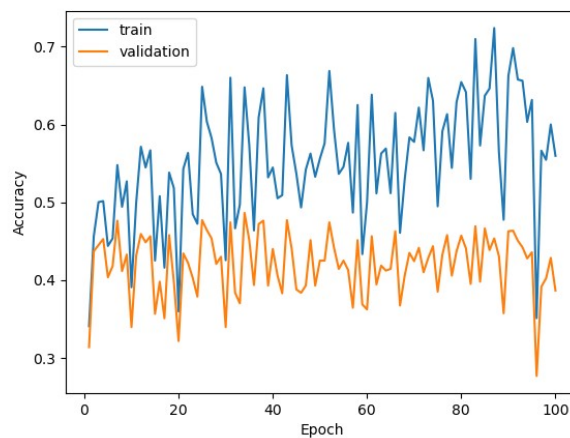
Group 52

Gonalo Rua (102604), Joo Gouveia (102611)

All the work was carried out by both members of the group, always together when trying to solve the problems presented in the different questions (including the ones that involved programming), and discussing the answer for them.

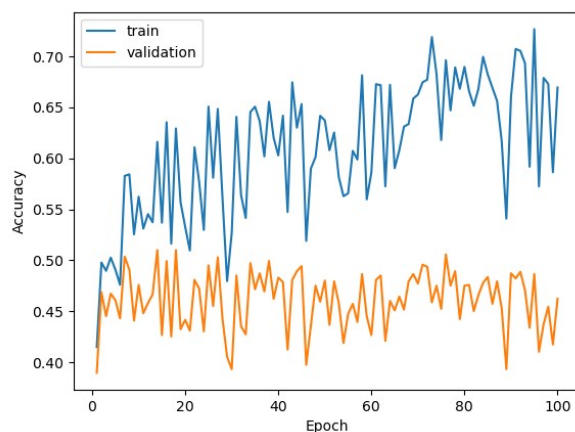
Question 1

1. (a) After implementing the code for the Perceptron class and running the command “python hw1-q1.py perceptron -epochs 100”, we obtained the following plot of the train and validation accuracies as a function of the epoch number:



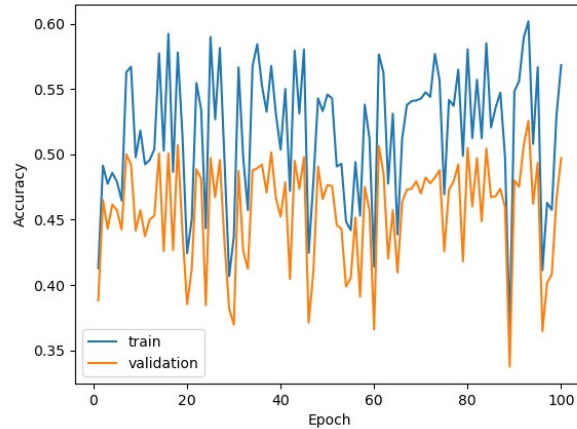
The final test accuracy obtained was 0.3743.

2. (a) After implementing the code for the LogisticRegression class (without L2 regularization) and running the command “python hw1-q1.py logistic_regression -epochs 100”, we obtained the following plot of the train and validation accuracies as a function of the epoch number:



The final test accuracy obtained was 0.4597.

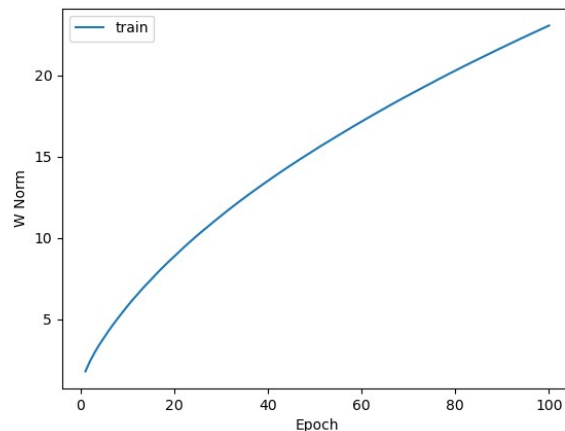
2. (b) After implementing the code for the LogisticRegression class (with L2 regularization) and running the command “python hw1-q1.py logistic_regression -epochs 100”, we obtained the following plot of the train and validation accuracies as a function of the epoch number:



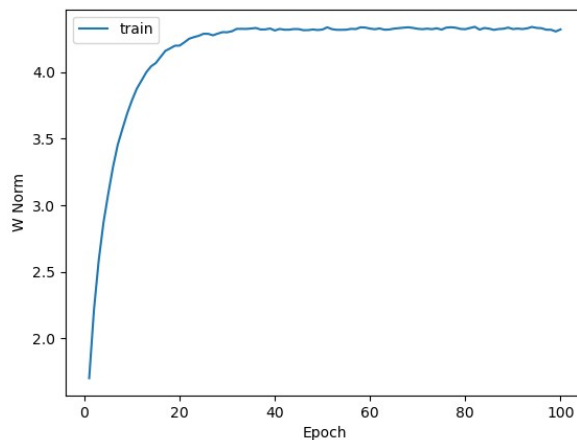
The final test accuracy obtained was 0.5053.

When comparing the regularized and non-regularized versions of the algorithm, we observe that the train and validation accuracies are more closely related in the regularized version. This is because regularization helps to control the complexity of the model by penalizing large weights, effectively avoiding overfitting to the training data, resulting in a model that is less likely to capture noise or overly specific patterns from the training set, leading to better generalization.

2. (c) The non-regularized version of the logistic regression classifier presents the following plot of the L2-norm of the weights as a function of the number of epochs:



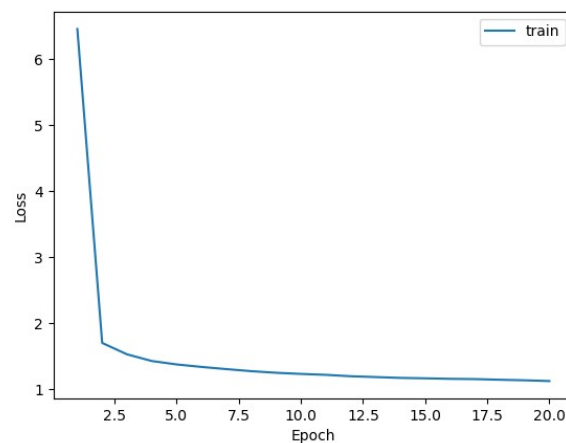
While the regularized version presents the following one:



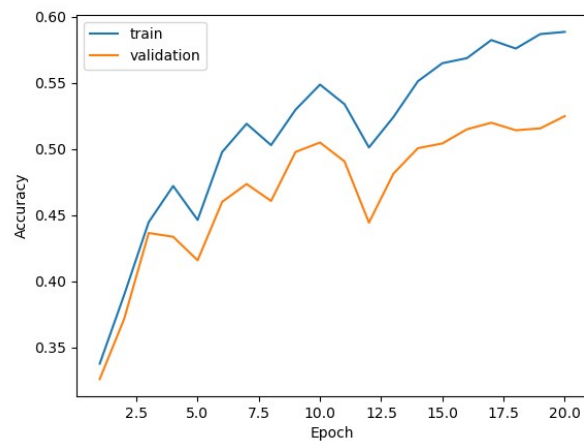
Comparing the two plots we verify that in the regularized version, the L2-norm of the weights increases initially but then stabilizes, in contrast to the non-regularized version, which keeps steadily increasing. This is expected because the L2 regularization penalizes large weights, achieving a balance between minimizing the loss on the training data and keeping the weights small to avoid overfitting. On the other hand, the non-regularized version weights grow without bound, aiming to perfectly fit the training data.

2. (d) L1 regularization, in addition to providing smaller weights, also promotes more sparse weights, meaning many of the weights will become zero, which results in a simpler model by selecting only a subset of the most important features.

3. (a) After implementing the MLP class, and running the command with the base command “python hw1-q1.py mlp”, we obtained the following plot of the train loss as a function of the epoch number:



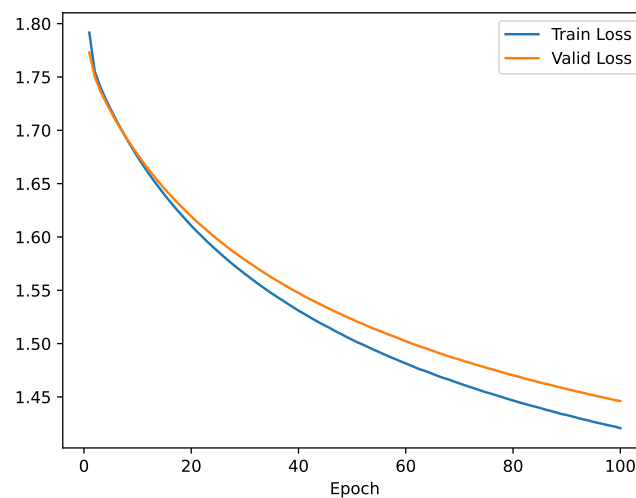
And the following plot of the train and validation accuracies as a function of the epoch number:



The final test accuracy obtained was 0.5337.

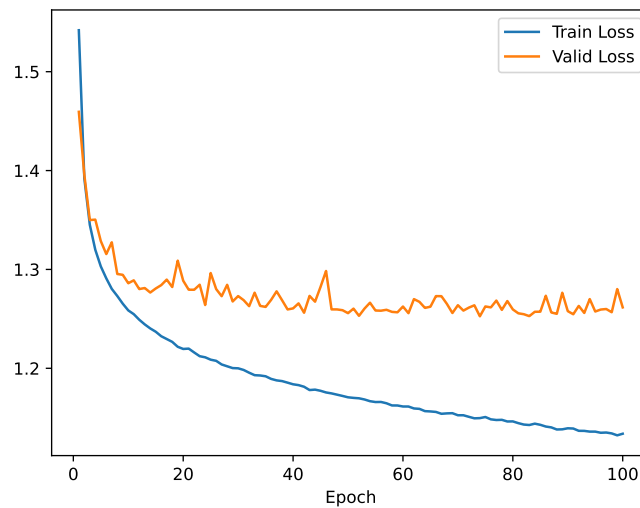
Question 2

1. After implementing the linear model using logistic regression and stochastic gradient descent for training, we executed the command “python hw1-q2.py logistic_regression -batch_size 32 -l2_decay 0.01 -epochs 100 -learning_rate 0.00001” and obtained the plot:



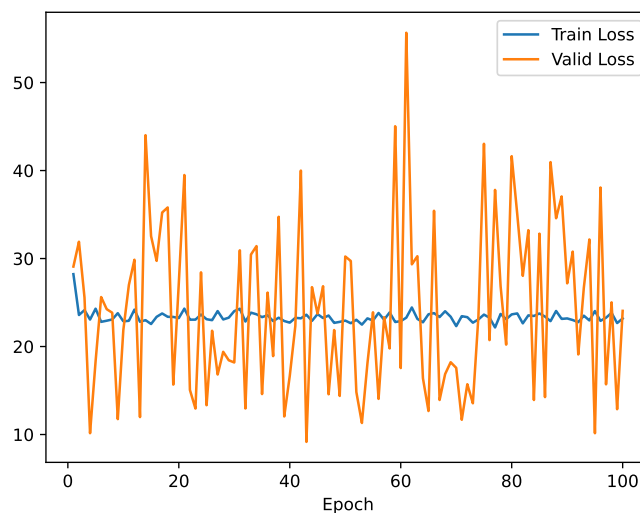
The final test accuracy obtained was 0.4623 and the final validation accuracy obtained was 0.4694.

We did the same thing for learning_rate equal to 0.001:



The final test accuracy obtained was 0.5247 and the final validation accuracy obtained was 0.5264.

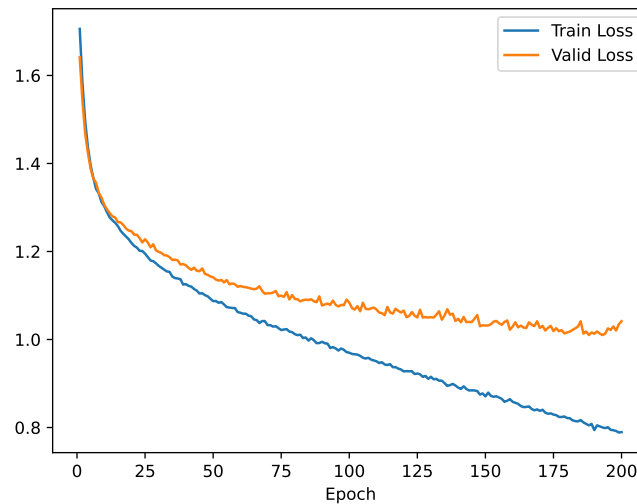
And for learning rate equal to 0.1:



The final test accuracy obtained was 0.4057 and the final validation accuracy obtained was 0.4152.

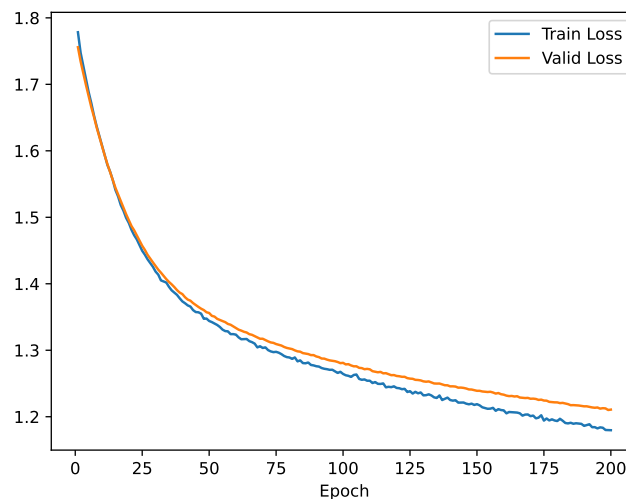
From the obtained data we observe that different learning rates result in different validation loss curves. While with the lowest learning rate of 0.00001 the validation loss decreases very slowly and consistently, indicating a slow but stable learning, with the highest learning rate of 0.1 we see erratic fluctuations in validation loss, which demonstrates a failure to converge due to overshooting, that leads to poor accuracy. With the more moderate approach of 0.001 as the learning rate, we see the validation loss decreasing and then stabilizing, resulting in the highest accuracy of 0.5264, making it the optimal learning rate.

2. (a) After implementing the feed-forward neural network, we executed the command “python hw1-q2.py mlp” in order to use the default hyperparameters, and obtained the plot:



The final test accuracy obtained was 0.6010 and the final validation accuracy obtained was 0.6011. The training took 2 minutes and 11 seconds.

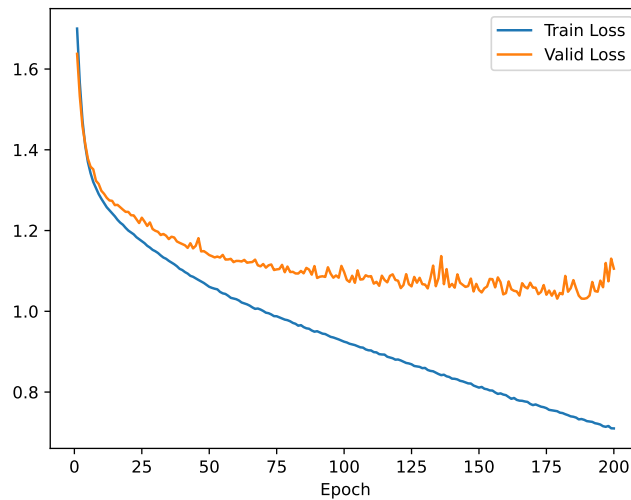
Then, we modified the command to use 512 as the batch_size, “python hw1-q2.py mlp -batch_size 512”, and obtained the plot:



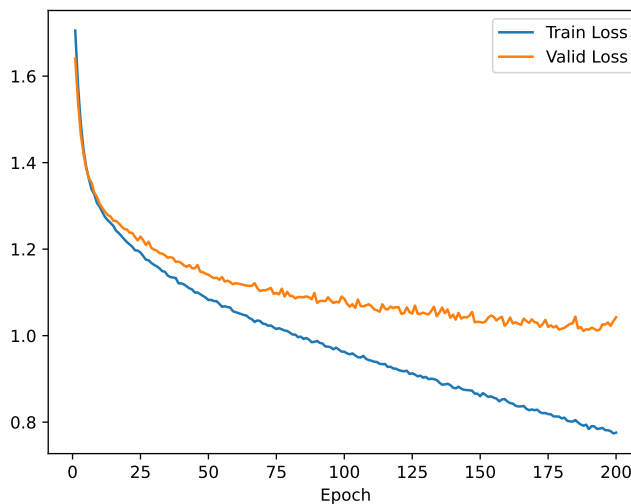
The final test accuracy obtained was 0.5443 and the final validation accuracy obtained was 0.5306. The training took 1 minute and 10 seconds.

We observe that the version that used a batch size of 64 achieves higher accuracy at the cost of increased training time. This is because a lower batch size means that the parameters update more frequently (more iterations per epoch), resulting in the capture of more details in the data. On the other hand, the version with 512 batch size shows signs of underfitting due to less frequent parameter updates, resulting in poorer performance.

2. (b) To train the model with the dropout setting as 0.01 we ran the command “python hw1-q2.py mlp -dropout 0.001”, and obtained the plot:

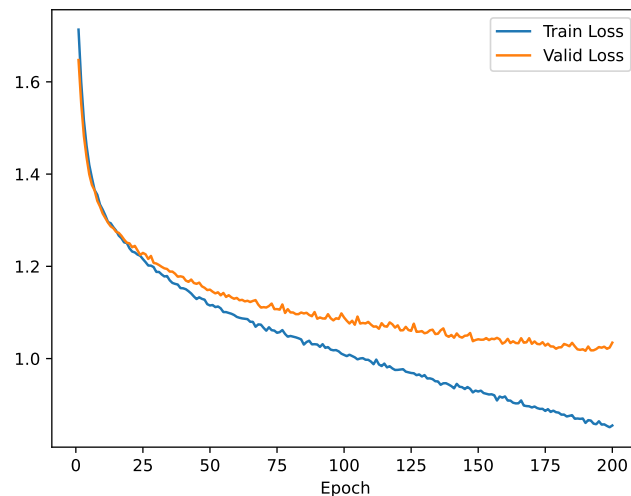


The final test accuracy obtained was 0.5853 and the final validation accuracy obtained was 0.5805. Then, we did the same thing, but now with dropout setting as 0.25, and obtained the plot:



The final test accuracy obtained was 0.6010 and the final validation accuracy obtained was 0.6026.

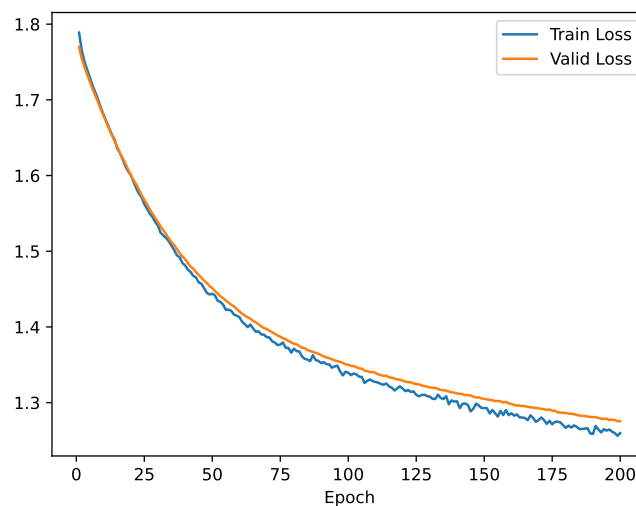
Finally, we did the same, but with dropout setting as 0.5, and obtained the plot:



The final test accuracy obtained was 0.6030 and the final validation accuracy obtained was 0.6090.

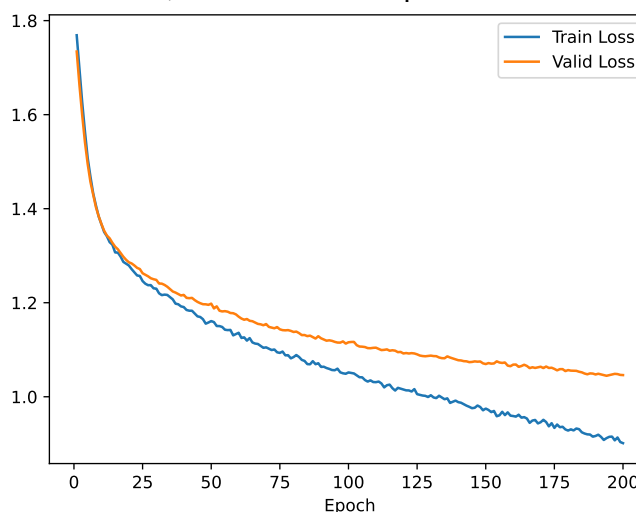
Dropout is a form of regularization technique that aims to avoid overfitting. The small dropout of 0.01 only introduces a small amount of regularization, which can be observed in the validation loss curve as a sign of overfitting, as the model relies more on specific neurons during training. With dropout of 0.25, we see less signs of overfitting in the curves and the model performs better with higher levels of final accuracy. Finally, with dropout of 0.5, although we obtain better final accuracy, we can see more noise in the train loss curve, as it introduces significantly more randomness during the training, dropping more nodes at each training step. Despite this noise, dropout still improves generalization as we can see an even greater reduction in overfitting.

2. (c) To train the model with the batch size 1024 and momentum as 0.0 we ran the command “python hw1-q2.py mlp -batch_size 1024 -momentum 0.0”, and obtained the plot:



The final test accuracy obtained was 0.5200 and the final validation accuracy obtained was 0.5014.

For the same batch size and momentum equal to 0.9, we executed the command “python hw1-q2.py mlp -batch_size 1024 -momentum 0.9”, and obtained the plot:



The final test accuracy obtained was 0.6000 and the final validation accuracy obtained was 0.6026.

Momentum aims to reduce the update in directions with changing gradients and increase the update in directions with stable gradient, accelerating convergence. This can be seen when comparing the two results above, where momentum 0.9 gives us a faster optimization, with higher final validation and test accuracy in the same number of epochs. However, this also translates in more overfitting to the training data, as we can observe in the bigger discrepancy between the train and validation loss curves. As the training loss decreases more aggressively with momentum, the validation loss does not decrease at the same rate.

Question 3

1.

①

$$h = g(wx + b)$$

$$g(z) = z(1-z) = z - z^2$$

$$h = (wx + b) - (wx + b)^2$$

If we do this neuron by neuron we get!

$$h_i = (w_i x + b_i) - (w_i x + b_i)^2$$

$$(w_i x + b_i)^2 = \sum_{j=1}^D \sum_{\ell=1}^D w_{ij} w_{i\ell} x_j x_\ell + 2b_i \sum_{j=1}^D w_{ij} x_j + b_i^2$$

$$h_i = (w_i x + b_i) - \sum_{j=1}^D \sum_{\ell=1}^D w_{ij} w_{i\ell} x_j x_\ell - 2b_i \sum_{j=1}^D w_{ij} x_j - b_i^2 + b_i$$

$$= \underbrace{\sum_{j=1}^D w_{ij} x_j}_{\text{linear terms}} - \underbrace{\sum_{j=1}^D \sum_{\ell=1}^D w_{ij} w_{i\ell} x_j x_\ell}_{\text{quadratic terms}} + \underbrace{(-b_i^2 + b_i)}_{\text{Constant term}}$$

$$\phi(x) = \begin{bmatrix} 1 & x_1 & x_2 & \dots & x_D & x_1^2 & x_1 x_2 & \dots & x_D^2 \end{bmatrix}^T \in \mathbb{R}^{\frac{(D+1)(D+2)}{2}}$$

$\phi(x) \in \mathbb{R}^{\frac{(D+1)(D+2)}{2}}$ because there are D linear terms + $\frac{D(D+1)}{2}$ quadratic terms + 1 constant term: $\frac{D(D+1)}{2} + D + 1 = \frac{D^2 + D + 2D + 2}{2} = \frac{D^2 + 3D + 2}{2} = \frac{(D+1)(D+2)}{2}$

$A_{ij} = [a_{ij}]$, where a_{ij} are coefficients of $\phi_j(x)$ in the expansion h_i
 $A \in \mathbb{R}^{K \times \frac{(D+1)(D+2)}{2}}$, where K is the number of neurons.

2.

②

$$h = A_{\Theta} \phi(x)$$

$$\hat{y} = v^T h + v_0 = v^T (A_{\Theta} \phi(x)) + v_0 = \underbrace{(v^T A_{\Theta})}_{\in \mathbb{R}^{1 \times \frac{(D+1)(D+2)}{2}}} \phi(x) + v_0$$

$$c_{\Theta} = \begin{bmatrix} v_0 + v^T A_{\Theta}[:, 1] \\ v^T A_{\Theta}[:, 2] \\ \vdots \\ v^T A_{\Theta}[:, \frac{(D+1)(D+2)}{2}] \end{bmatrix}$$

R.: No, this is not a linear model in terms of the original parameters Θ . ~~Although~~ Although it is linear in the feature space $\phi(x)$, the hidden layer has activation $g(z) = z(1-z)$, which introduces non-linearity in ~~weights~~ $W, b \in \Theta$.

3.

$$\hat{y} = C_{\oplus}^T \phi(x)$$

$$C_{\oplus} = \begin{bmatrix} v_0 + v^T A_{\oplus}[:, 1] \\ v^T A_{\oplus}[:, 2] \\ \vdots \\ v^T A_{\oplus}[:, 3] \end{bmatrix} \leftarrow \text{constant term}$$

If $A_{\oplus}$ is singular, we know that it is ϵ -close to a non-singular matrix, and we can perturb it slightly to get the non-singular matrix $A'_{\oplus}$:

$$A'_{\oplus} = A_{\oplus} + \epsilon I$$

Then we can use QR decomposition, which gives us:

$$A'_{\oplus} = QR$$

And approximate c as:

$$c^T = v^T QR$$

where the constant term is directly set in c .

To obtain $\hat{y}$ from c :

- ① If $A_{\oplus}$ is singular, perturb it by ϵ to make it ^{into the} non-singular matrix $A'_{\oplus}$.
- ② Compute the decomposition $A'_{\oplus} = QR$.
- ③ Solve for v : $v^T = c^T R^{-1} Q$, manually setting the constant term in c .
- ④ Use v , Q , and R to reconstruct W , b .

4.

$$\textcircled{4} \quad D = \{(x_n, y_n)\}_{n=1}^N, \quad N > \frac{(D+1)(D+2)}{2}$$

$$X \in \mathbb{R}^{N \times \frac{(D+1)(D+2)}{2}}, \text{ whose rows are } \{\phi(x_n)\}_{n=1}^N$$

$$L(C_{\oplus}; D) = \frac{1}{2} \sum_{n=1}^N (\hat{y}(x_n; C_{\oplus}) - y_n)^2 = *$$

Squared loss

$$\hat{y}(x; C_{\oplus}) = C_{\oplus}^T \phi(x)$$

$$* = \frac{1}{2} \sum_{n=1}^N (C_{\oplus}^T \phi(x_n) - y_n)^2$$

$$\hat{C}_{\oplus} = \min_{C_{\oplus} \in \mathbb{R}^{\frac{(D+1)(D+2)}{2}}} \frac{1}{2} \sum_{n=1}^N (C_{\oplus}^T \phi(x_n) - y_n)^2 = (X^T X)^{-1} X^T y$$

Proof

$$\sum_{n=1}^N (C_{\oplus}^T \phi(x_n) - y_n)^2 = \|X C_{\oplus} - y\|^2 = \|y - X C_{\oplus}\|^2$$

$$0 = \nabla_{C_{\oplus}} \|y - X C_{\oplus}\|^2 \Leftrightarrow 0 = \nabla_{C_{\oplus}} (C_{\oplus}^T X^T X C_{\oplus} - 2 C_{\oplus}^T X^T y + \|y\|^2) \Leftrightarrow$$

$$\Leftrightarrow 0 = 2 X^T X C_{\oplus} - 2 X^T y \Leftrightarrow \hat{C}_{\oplus} = (X^T X)^{-1} X^T y$$

R.: Usually, in feedforward neural networks, the loss function is non-convex because the hidden layers introduce non-linear dependencies ~~on the parameters~~ on the parameters. Because in this problem we can reparametrize the network as a linear model in the feature space $\phi(x)$, minimizing the squared loss has a closed-form solution, because the loss function is convex.